

Lecture-7

ALTER TABLE Statement

- The **ALTER** command is used to **change the structure or design of an existing table** in your database.

What ALTER TABLE lets you do:

1. **Add a column:** You can introduce a brand new column to a table.
2. **Modify a column:** You can change the type of data a column stores (its data type) or adjust the size of the data it can hold.
3. **Rename a column:** You can change the name of an existing column.
4. **Drop a column:** You can permanently remove a column from the table.
5. **Rename the table:** You can change the name of the entire table.

Closer Look at Modifying Tables:

- Adding Columns
 - You use the **ALTER TABLE** command along with **ADD** to put a new column into a table.
 - *Example:* `ALTER TABLE table_name ADD column_name datatype;`
 - *Example:* To add a "Publisher" column to the "Authors" table that stores text up to 200 characters: `ALTER TABLE Authors ADD(Publisher VARCHAR(200));`
 - **Constraints (Rules):** Constraints are important theoretical concepts. They are rules applied to columns to make sure the data is accurate, consistent, and reliable. Common examples include **Not Null** (a value must be present), **Primary Key**, and **Foreign Key**.
 - **Adding Multiple Columns:** You can add several new columns at once by separating their definitions with commas inside the parentheses after the **ADD** clause.
 - *Example:* Adding "Department," "Gender," and "dob" (date of birth) columns: `ALTER TABLE Authors ADD(Department VARCHAR(60), Gender VARCHAR(60), dob DATE);`
- Modifying Existing Columns

- The **ALTER** command, usually combined with **MODIFY**, lets you adjust an existing column.
- **Changing Data Type or Size:** You can change the data type of a column or increase its maximum size.
 - *Example:* Changing the **dob** column from a **date** type to a **varchar** (text) type that holds 20 characters: `ALTER TABLE Authors MODIFY dob varchar(20);` .
 - *Example:* Changing the size limit of the **Department** column from 60 characters to 100 characters: `ALTER TABLE Authors MODIFY Department varchar(100);` .
 - **Changing Nullability:** You can change a column that currently allows empty entries (**NULL**) to one that requires an entry (**NOT NULL**).
 - *Example:* Making the **LastName** column mandatory (not null): `ALTER TABLE Authors MODIFY LastName varchar(50) NOT NULL;` .
- Renaming a Column
 - To rename a column, you use the **CHANGE COLUMN** clause.
 - **Important Requirement:** When you rename a column, you must provide its full new definition (including its data type and whether it is **NULL** or **NOT NULL**), even if those properties haven't changed.
 - *Example:* Renaming the **LastName** column to **Surname** : `ALTER TABLE Authors CHANGE COLUMN LastName Surname VARCHAR(50) NOT NULL;` .
- Dropping a Column

You can delete a column using the **DROP COLUMN** clause.

 - *Example:* Removing the **dob** column from the "Authors" table: `ALTER TABLE Authors DROP COLUMN dob;` .
- Renaming the Table

You can rename the entire table structure using **RENAME TO** .

 - *Example:* Renaming the "Authors" table to "AuthorsInfo": `ALTER TABLE Authors RENAME TO AuthorsInfo;` .

DCL Commands (Data Control Language)

- DCL commands are essential for **database security** in environments where multiple users access the data.

What DCL Does (Theoretical Explanation):

DCL commands control **user access** to the database. They help restrict users from seeing or changing data they shouldn't be able to.

- **Authority:** Only the Database Administrator (DBA) or the owner of the database objects has the authority to grant or remove these permissions.
- The two main DCL commands are **GRANT** and **REVOKE**.

The GRANT Command

The **GRANT** command is used to **give a user permission** to carry out specific tasks (like reading or changing data) on the database.

Types of Permissions (Privileges):

- **ALL:** Gives the user every available privilege.
- **SELECT:** Allows the user to read (view) data from tables or views.
- **INSERT:** Allows the user to add new data to a table.
- **UPDATE / DELETE:** Allows the user to change or remove existing data.
- **CREATE / DROP / ALTER:** Allows the user to create new tables, delete database objects, or change the structure of existing tables, respectively.

The WITH GRANT OPTION (Theoretical Detail):

When you grant a user permission using the **WITH GRANT OPTION**, you are giving them the power to **pass that same permission along to other users**.

- **A Cautionary Note:** You must use this option carefully. If User A grants SELECT rights *with grant option* to User B, and User B then grants those rights to User C, if you later **REVOKE** the rights from User B, **User C will still keep their permissions**.

```
GRANT privilege_name
ON object_name
TO {user_name | PUBLIC | role_name}
[WITH GRANT OPTION];
```

```
GRANT select, insert, update, delete
ON Employee
TO user1
WITH GRANT OPTION;
```

The REVOKE Command

The **REVOKE** command is used to **take away (cancel) user access rights** or privileges that were previously granted.

- **Basic Function:** It removes a privilege (`privilege_name`) on a specific database object (`object_name`) from a user (`user_name` or `role_name`).
- *Example:* Removing the ability to read data from the "Employee" table for user1: `REVOKE SELECT ON Employee FROM user1;` .

Important Rules for REVOKE:

1. **Source of Grant:** You can only use **REVOKE** to take away privileges that you originally granted yourself.
2. **Multiple Grants:** If a user received the same privilege from several different sources, you must revoke the privilege from *all* those sources before the user truly loses that access.
3. **Revoking Grant Option:** If you want to remove a user's ability to pass permissions to others, you must specifically revoke the `grant option` as well.

5. Roles

Roles are a theoretical tool used for easier management of user privileges.

What Roles Are:

- A role is simply a **bundle or collection of different privileges** (permissions) grouped together.

Why Use Roles (Theoretical Explanation):

- In databases with many users, it can be inefficient and difficult for administrators to grant or revoke individual permissions for every single person.
- Roles simplify this process greatly, allowing administrators to assign an entire set of needed permissions (the "Role") in just one step.

- If the required permissions for a role change, the administrator only has to update the privileges assigned to the Role itself. These changes are then **automatically applied** to all users assigned to that Role.

```
REVOKE privilege_name  
ON object_name  
FROM {user_name | PUBLIC | role_name}
```

```
REVOKE select, insert, update, delete  
ON Employee  
FROM user1;
```